

Authentication Redirect Loop — Investigation & Fix Report

Date: September 9, 2026 **Project:** Connectify **Summary:** After migrating from a mock API to the real backend, users were logged out and redirected back to `/login` within seconds of a successful login. Root cause was traced through three separate bugs across the frontend and backend. This report documents each, the fix applied, and remaining follow-up items.

Symptom

User logs in successfully; the app briefly shows the authenticated state. Within moments, the app performs what looks like a full page reload and lands back on the login (or landing) page. A network request appears briefly in DevTools and then disappears before it can be inspected.

Root Cause Chain

The issue was not one bug but three, each masking the next until fixed in sequence.

1. Missing error handling in `AuthProvider`'s session-restore effect

File: `context/authContext/AuthProvider.tsx`

The effect that runs on app load to restore a session had no `try/catch/finally`. Any thrown error (e.g. a 401 from an expired token against the real backend, where the mock API never rejected requests) stopped execution before `setIsLoading(false)` ran, leaving the app stuck on a loading spinner indefinitely.

Fix: Wrapped the effect body in `try/catch/finally`, ensuring `isLoading` is always cleared and the user/profile state is reset to a clean "logged out" state on failure.

2. Response envelope mismatch in `services/authApi.ts`

Files: `services/authApi.ts`

The backend consistently wraps responses as `{ message, data, requestId }`. Several frontend functions (`getCurrentUser`, `getProfile`, and others) used a defensive fallback chain — `data?.user ?? data?.data?.user ?? data` — that didn't match this shape. For `GET /api/v1/auth/me`, this caused `getCurrentUser()` to return the *entire response envelope* instead of the user object, since neither `.user` nor `.data.user` existed.

That malformed "user" object had no `.id`, so the next call in `AuthProvider` — `getProfile(current.id)` — was made as `GET /api/v1/profiles/undefined`.

Fix: Updated the extraction logic to `data?.data` to match the backend's actual envelope shape, starting with `getCurrentUser` and `getProfile`. Added a guard in `AuthProvider` (`if (current?.id)`) so a similar future mismatch degrades gracefully instead of firing a malformed request.

Follow-up (not yet applied everywhere): The same fragile fallback pattern still exists in `getMyProfile`, `saveProfile`, `getProfileById`, `getWhoLikedMe`, `getMatches`, `likeUser`, `getLikedByMe`, and `getConversations`. These haven't caused visible bugs yet only because those code paths haven't been exercised against the real backend. Recommend sweeping all of them to `data?.data` for consistency.

3. Missing auth middleware on the profile discovery route (confirmed root cause)

Files: `routes/profiles.routes.ts`, `profiles.controller.ts`

`DiscoveryPage` (the page a user lands on immediately after login) calls `GET /api/v1/profiles` on mount to load discoverable profiles. The route was registered without `authMiddleware`:

```

ts
profilesRouter.get('/', profilesController.list)
```

However, the `list` controller assumes authentication already happened and checks `request.user`:

```

ts
if (!request.user) {
  return response.status(401).json({ message: 'Authentication required' });
}
```

Because `authMiddleware` never ran on this route, `request.user` was always `undefined`, so this endpoint returned **401 on every request, regardless of token validity**. The frontend's global axios response interceptor treats any 401/403 as "not authenticated": it clears the stored token and performs a hard `window.location.replace('/login')`. This full-page reload is what caused the app to appear to "log in, flash, and bounce back" — and why the network request appeared to vanish (a page reload cancels in-flight requests and clears the Network panel unless "Preserve log" is enabled).

Fix:

```

ts
profilesRouter.get('/', authMiddleware, profilesController.list)
```

This was the fix that resolved the redirect loop.

Additional Issues Found (Not Blocking, Recommended Follow-Up)

While reviewing `profiles.controller.ts`, two more issues were identified in the `list`, `getById`, and

`getMe` handlers:

Dead/duplicated code in `list`: The handler calls `profilesService.listProfiles()`, immediately sends a hardcoded `501 Not Implemented` response, and *then* contains a second, unreachable `try/catch` block with the real implementation below it. This appears to be a leftover stub that was never removed after the real implementation was added. As written, the second block would also throw a "headers already sent" error if it were ever reached.

`getById` and `getMe` follow the same pattern — they call the real service function, discard the result, and unconditionally return `501`. Any frontend code depending on `GET /profiles/:id` or `GET /profiles/me/profile` returning real data is currently getting a stub response instead, silently.

`getMe` has no `authMiddleware` despite being a "my profile" endpoint that should require authentication.

Recommendation: Remove the leftover 501 stub blocks in `list`, `getById`, and `getMe`, and add `authMiddleware` to `getMe` (and to `getById` if profile detail views should require login).

Verification

After applying fix #3 (adding `authMiddleware` to the `/profiles` route):

Login completes and the user remains authenticated.

`POST /api/v1/auth/login` → 200

`GET /api/v1/auth/me` → 200

`GET /api/v1/profiles/:id` → 200

`GET /api/v1/profiles?...` (discovery) → 200, no reload

Summary Table

#	Layer	File	Issue	Status
1	Frontend	<code>AuthProvider.tsx</code>	No error handling on session-restore effect; <code>isLoading</code> never cleared on failure	✅ Fixed
2	Frontend	<code>authApi.ts</code>	Response envelope fallback chain didn't match backend shape; <code>getCurrentUser</code> returned malformed data	✅ Fixed (<code>getCurrentUser</code> , <code>getProfile</code>); ⚠️ same pattern remains in other functions
3	Backend	<code>profiles.routes.ts</code>	<code>GET /profiles</code> missing <code>authMiddleware</code> , causing guaranteed 401 → hard redirect loop	✅ Fixed (root cause)
4	Backend	<code>profiles.controller.ts</code>	Dead code / stub 501 responses left in <code>list</code> , <code>getById</code> , <code>getMe</code>	⚠️ Open — recommended follow-up

#	Layer	File	Issue	Status
5	Backend	<code>profiles.routes.ts</code>	<code>getMe</code> missing <code>authMiddleware</code>	 Open — recommended follow-up